FINANCIAL WEATHER

AN INTELLIGENT FINANCIAL STRESS DETECTION SYSTEM

USING USER SPENDING BEHAVIOR

Technical Documentation • Setup Guide • User Manual

Prepared by: Harith Malan

August 2026

A calm, explainable read on spending behavior — not a diagnosis.

Table of Contents

PART I — Project Documentation

1. Introduction & Objectives 4
2. System Architecture 4
3. Technology Stack 5
4. Dataset & Data Artifacts 6
5. Data & ML Processing Pipeline 6
6. Fairness, Ethics & Responsible Design 7
7. Feature Overview 8
8. Evaluation: Model Comparison 10
9. Known Decisions & Limitations 13

PART II — Setup Guide

10. Prerequisites 15
11. Backend & Frontend Setup 15
12. Running the Application 15
13. Troubleshooting 16

PART III — User Manual

14. Getting Started 18
15. Filling In Your Information 18
16. Understanding Your Result 18
17. What Stood Out & Gentle Suggestions 18
18. Checking Another Scenario 19
19. Accessibility & Privacy Notes 19

PART I

Project Documentation

1. Introduction & Objectives

2. System Architecture

3. Technology Stack

4. Dataset & Data Artifacts

5. Data & ML Processing Pipeline

6. Fairness, Ethics & Responsible Design

7. Feature Overview

8. Evaluation: Model Comparison

9. Known Decisions & Limitations

1. Introduction & Objectives

Most personal finance apps are reactive — they show a person what they already spent, after the fact. This project takes a predictive approach instead: it classifies a user's financial stress level as Low, Moderate, or High from their spending behavior, using machine learning trained on income, expenditure, debt, savings, and behavioral indicators rather than a single income-vs-expenditure rule.

The central argument motivating this design is that financial stress can exist even when someone technically spends within their income — driven instead by debt load, low savings, or volatile discretionary spending. A rule as simple as “expenditure exceeds income” misses that case entirely, which is why this system compares several machine learning models against a richer feature set instead.

Objectives

- **Behavioral classification.** Compare five machine learning models (Logistic Regression, Decision Tree, Random Forest, SVM, XGBoost) rather than committing to one algorithm or a fixed rule.
- **Multi-factor scoring.** Engineer ratio-based features (debt-to-income, savings ratio, discretionary ratio, spending volatility, and more) rather than relying on income alone.
- **Explainability.** Justify every prediction with feature importance, SHAP values, and a plain-language summary of the top contributing factors.
- **Calm, non-stigmatizing UX.** Present results through a “financial weather” metaphor (Clear / Changeable / Stormy) instead of alarming red gauges or clinical language.
- **Reproducibility.** Validate every pipeline stage with an automated self-check script that prints PASS/FAIL against concrete expectations, not just a description of what was built.

Scope of this Document

This document describes the working software artifact as built. The system's associated academic research proposal describes primary data collection through an online questionnaire using a validated instrument (the CFPB Financial Well-Being Scale) and a full year-long research timeline. For this software artifact, a documented synthetic dataset was used instead, generated to match the proposal's exact feature list and labeling methodology — see Section 4 and Section 9 for the full rationale and its limitations, stated explicitly rather than left as an unstated gap.

2. System Architecture

The application runs as a single Python process: a FastAPI backend that both serves the prediction API and hosts the static frontend files, with no separate frontend server, build step, or database. The offline training pipeline (data generation through model selection) runs separately and produces the artifacts the live API depends on.

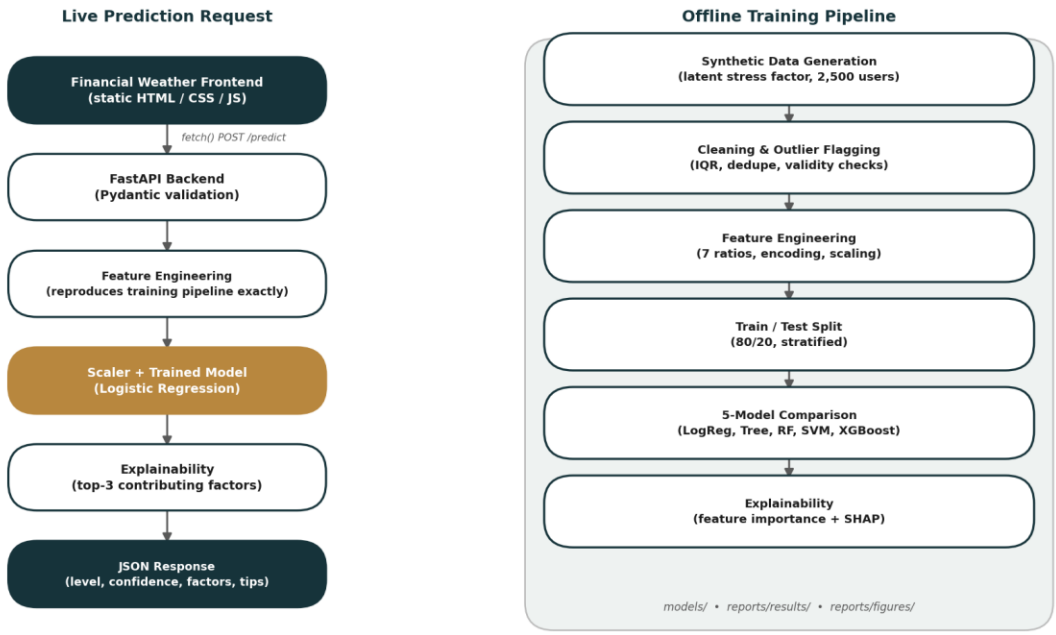


Figure 1 — The live prediction request path (left) and the offline training pipeline that produces its model artifacts (right).

Project Structure

- **data/{raw,processed,interim}/ —** the synthetic dataset at each pipeline stage.
- **src/{data,features,models,evaluation,app}/ —** cleaning, feature engineering, model training, explainability, and the FastAPI app itself.
- **models/ —** trained model files (.joblib), the fitted scaler, and the selected best-model metadata.
- **reports/{figures,results}/ —** the model comparison table, explainability charts, and written evaluation.
- **scripts/ —** the reusable self-check/validation scripts run after every pipeline stage.
- **tests/ —** the pytest suite (unit, functional, and API validation/security tests).

3. Technology Stack

Layer	Technologies
Frontend	HTML5, CSS3 (custom design system), vanilla JavaScript, Three.js (loaded via CDN for the weather visualization, with a static-SVG fallback)
Backend	FastAPI (Python), Pydantic (request validation), Uvicorn
ML / Data	pandas, numpy, scikit-learn (Logistic Regression, Decision Tree, Random Forest, SVM), XGBoost, SHAP, joblib, matplotlib
Testing	pytest (66 automated tests), Playwright (headless-browser and screenshot verification), Node.js (JavaScript syntax checking only, not a runtime dependency)

No database, by design

Layer	Technologies
All state is file-based — the dataset lives in data/, model artifacts in models/. Every prediction request is stateless: nothing about a user's input is stored or persisted anywhere, consistent with the app's own “nothing here is stored or shared” notice.	

4. Dataset & Data Artifacts

Rather than a database, the system is backed by a synthetic, questionnaire-style dataset of 2,500 users and 35 columns, generated from a shared hidden “stress factor” so that features correlate with the outcome the way real behavioral data would, without being a deterministic formula of it. The full generation methodology and every column's definition are documented in data_dictionary.md.

Column Group	Contents
Demographics	age, employment_status, dependents
Income & Expenditure	monthly_income, monthly_expenditure, essential_expenditure, non_essential_expenditure
Category Breakdown	10 categories: housing, utility, food, healthcare, education, transportation, shopping, entertainment, subscription, other
Debt & Commitments	debt_repayment, num_financial_commitments
Savings & Budget	savings, planned_expenditure, budget_deviation

Column Group	Contents
Transaction Behavior	transaction_frequency, avg_transaction_value, spending_volatility
Behavioral Indicators	impulse/online/unplanned purchase frequency, budget_exceed_frequency, can_save_regularly, balance_check_frequency, spending_change_during_difficulty
Target	financial_wellbeing_score (0–100, continuous) and financial_stress_level (Low / Moderate / High)

Target Label Definition

The CFPB Financial Well-Being Scale does not publish an official Low/Moderate/High cutoff — CFPB's own guidance treats the score as continuous and compares it against national percentile benchmarks. This project instead uses fixed, sample-independent score bands (≥70 Low, 40–69 Moderate, <40 High), consistent with the "good / fair / needs attention" style bands practitioners commonly use when reporting CFPB-scale results — a documented decision, not an arbitrary split of this project's own sample.

Resulting class distribution: Low 870 (34.8%), Moderate 1,132 (45.3%), High 498 (19.9%).

5. Data & ML Processing Pipeline

Every prediction depends on the same pipeline having run offline, end to end, with each stage validated before the next began:

1. Generate the synthetic questionnaire-style dataset from a shared latent stress factor (2,500 users)
2. Apply the target-variable label using fixed, CFPB-informed score bands
3. Clean the data: remove duplicates, check for invalid values, flag (not remove) statistical outliers via the IQR method
4. Engineer 7 ratio features (spending-to-income, savings, debt-to-income, discretionary, essential, subscription burden, spending volatility) and encode categorical fields
5. Produce both scaled and unscaled feature sets, saving the fitted scaler for reuse at prediction time
6. Perform one stratified 80/20 train/test split, applied identically across both feature sets
7. Train five candidate models with 5-fold stratified cross-validation
8. Exclude the deterministic leakage feature (financial_wellbeing_score) before scoring, since it was the value the label was derived from
9. Evaluate every model on the held-out test set: accuracy, precision, recall, F1 (macro and per-class), and ROC-AUC
10. Select the best model, balancing performance, interpretability, and inference speed
11. Generate explainability artifacts: a feature importance chart, a SHAP summary plot, and a reusable explain_prediction() helper
12. Serve predictions via FastAPI: raw form input → feature engineering reproduced exactly → scaler → model → explanation

13. Render the result in the Financial Weather frontend with a confidence breakdown and general suggestions

Final Model Selection

Logistic Regression was selected. It trailed the best-performing model (SVM) by roughly one point of F1-macro and High-stress recall — a gap within noise on a roughly 500-user test set — while offering meaningfully better interpretability, the highest ROC-AUC of all five models, and substantially faster inference, which matters directly for the project's own explainability objective.

Explainable AI

Because Logistic Regression's decision function is a known, linear combination of its inputs, feature importance is read directly from the model's coefficients, and SHAP values are computed against that same function rather than approximated. Every prediction's explanation names the three factors that contributed most to that specific user's result, in plain language (e.g. “High debt-to-income ratio”, “Ability to save regularly: No”).

6. Fairness, Ethics & Responsible Design

Because this system estimates something as personal as financial stress, several deliberate design choices sit alongside the modeling work itself:

- **Non-diagnostic framing.** Every response — in the API and the interface — includes an explicit disclaimer that this is an automated estimate based on spending patterns, not a financial or psychological diagnosis.
- **Non-alarming design.** The interface deliberately avoids red alert banners or traffic-light gauges. The High-stress state uses a muted clay tone rather than alarm-red, and its copy reads “worth addressing gently, not alarmingly” rather than a warning.
- **Recall-conscious model selection.** High-stress recall was reported and weighed explicitly during model selection, on the reasoning that under-flagging a high-stress user is a costlier error than an extra false alarm.
- **No data persistence.** There are no user accounts and no database. Every prediction is stateless — nothing entered into the form is stored or shared, stated directly to the user before they submit anything.
- **Synthetic-data transparency.** The dataset's synthetic origin and generation methodology are documented in full (Section 4, data_dictionary.md) rather than presented as real user data.

7. Feature Overview

A calm, single-page tool: fill in a short questionnaire, submit, and see a weather-themed read on your current financial stress level, with the reasoning behind it.

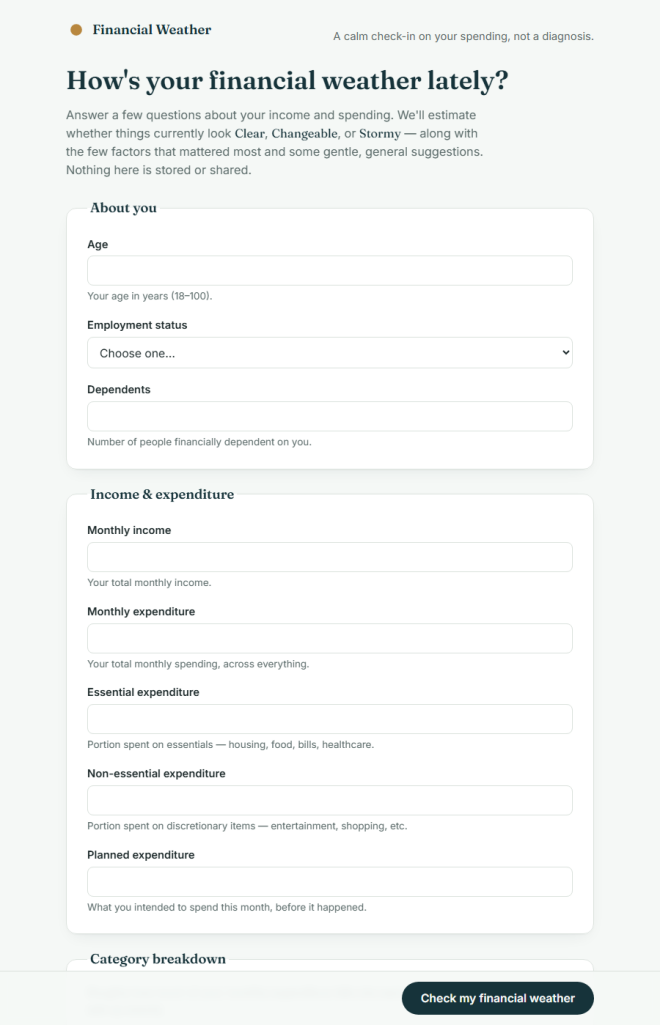


Figure 2 — The input form, grouped into five sections.

Grouped Input Form

Five sections — About you, Income & expenditure, Category breakdown, Debt & savings, Spending behavior — each visually distinct, with inline hints drawn from the same field definitions as the dataset itself.

Client-Side Validation

Mirrors the backend's Pydantic rules (age 18–100, income greater than zero, and so on) for instant feedback, with the backend still treated as the final authority.

Weather-Themed Result

Clear / Changeable / Stormy, replacing a clinical score or a traffic-light gauge with a calmer, non-stigmatizing metaphor.

Confidence Breakdown

The model's probability for all three classes, shown as simple labeled bars rather than a busy chart.

Plain-Language Explanation

The top three contributing factors for that specific prediction, plus general, non-professional-advice suggestions tied to those factors.

Accessibility Floor

Visible keyboard focus states, labelled form fields, full mobile-width usability down to 375px, and complete prefers-reduced-motion support that falls back to static icons and disables all choreographed animation.

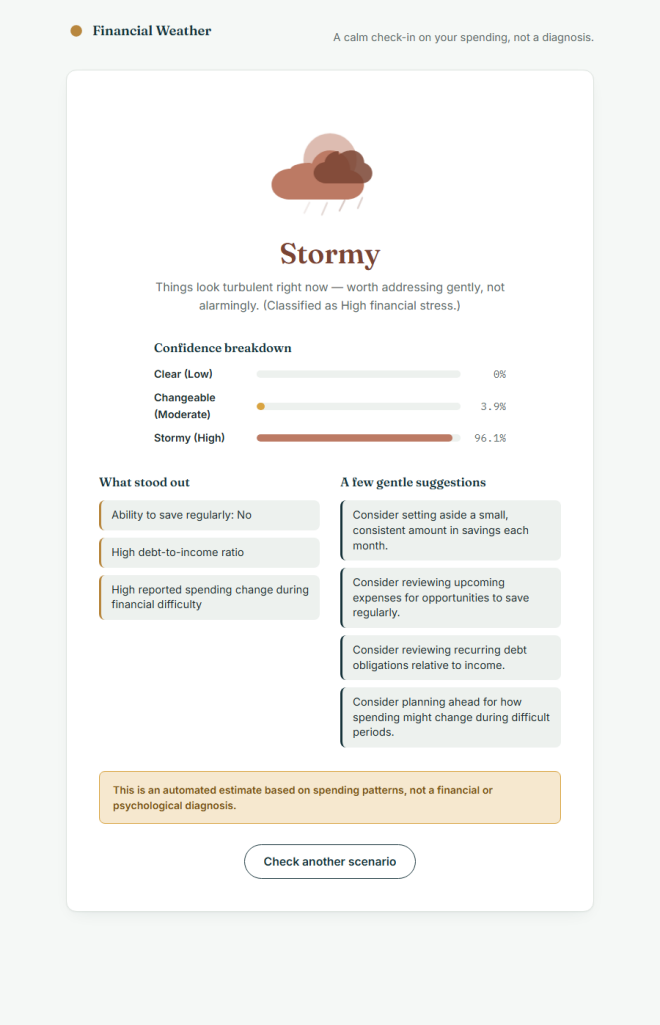


Figure 3 — A Stormy (High stress) result, with confidence breakdown, contributing factors, and suggestions.

8. Evaluation: Model Comparison

Five models were trained under identical conditions and compared on a held-out test set to select the model that would power the live API.

Methodology

2,500 synthetic users were split 80/20 (stratified by stress level), giving roughly 2,000 training and 500 test users. Logistic Regression and SVM trained on the scaled feature set; Decision Tree, Random Forest, and XGBoost trained on the unscaled set. All five were evaluated on the same held-out test users, with 5-fold cross-validation used during training to check consistency.

Model	Accuracy	F1 (macro)	Recall (High)	F1 (High)	ROC-AUC
Logistic Regression	0.766	0.765	0.69	0.746	0.915
SVM	0.778	0.776	0.70	0.753	0.910
Random Forest	0.764	0.761	0.66	0.733	0.907
XGBoost	0.762	0.761	0.69	0.742	0.904
Decision Tree	0.650	0.651	0.64	0.640	0.730

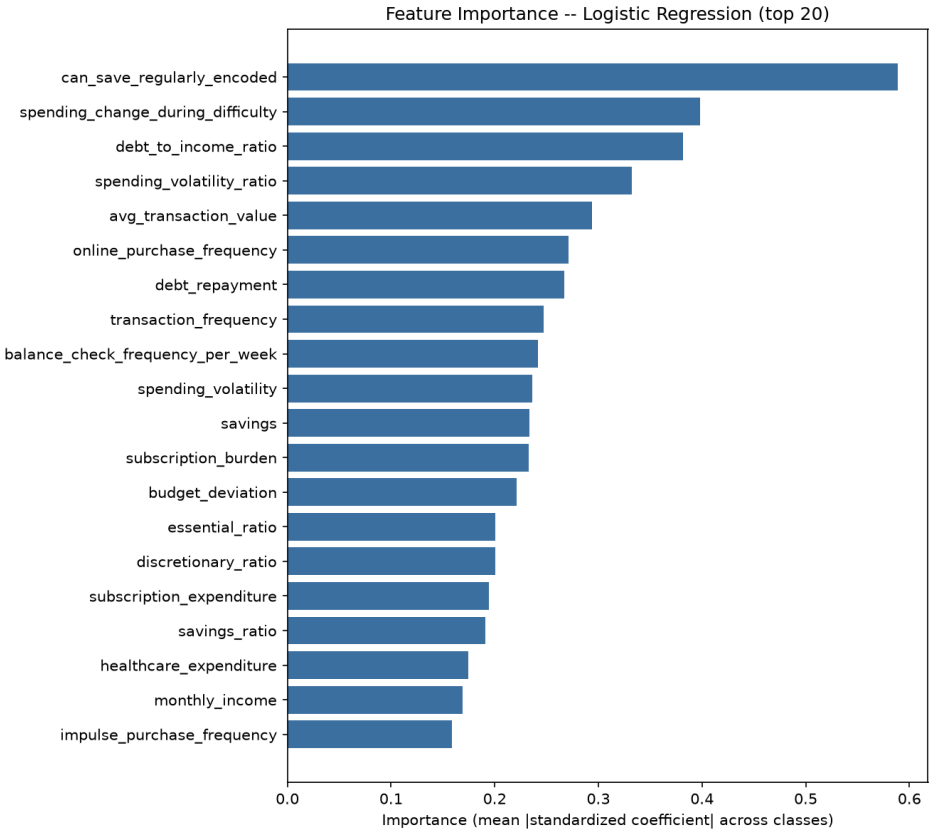


Figure 4 — Logistic Regression feature importance (top 20 features).

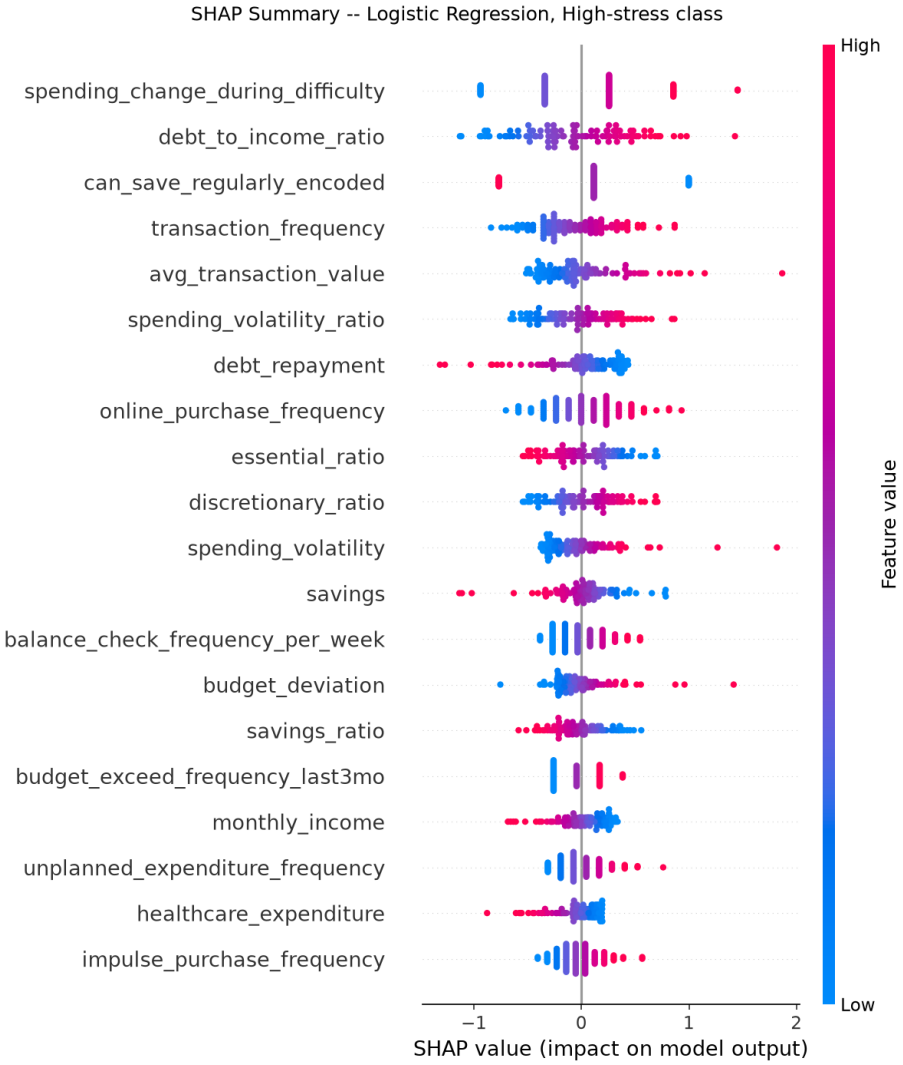


Figure 5 — SHAP summary for the High-stress class.

Verdict

SVM edges out Logistic Regression on F1-macro and High-stress recall by roughly one point — with around 100 High-stress users in the test set, that margin is within noise. Decision Tree is a clear outlier on the low end, consistent with a single tree overfitting relative to the regularized and ensemble alternatives. Income alone is a weak predictor (near-zero correlation with the underlying well-being score), while ability to save regularly, debt-to-income ratio, and spending volatility dominate feature importance — directly supporting the project's founding argument that behavioral features, not income, drive financial stress.

Evaluation Limitations

- Synthetic data: results describe how well the models learned the synthetic generation process's own patterns, not validated real-world behavior.
- A single fixed labeling threshold (Section 4) was used throughout; it was not compared against alternative thresholding schemes.

- Test set size (~500 users, ~100 High-stress) means model differences under roughly 1–2 points should be read as within noise, not a meaningful ranking.

9. Known Decisions & Limitations

The following are deliberate, documented decisions made during development — not silent gaps:

- **Synthetic data.** No primary questionnaire data was available at project start; a synthetic dataset was generated instead, matching the proposal's exact feature list, with the methodology fully documented (Section 4) rather than presented as real user data.
- **Target label thresholds.** Fixed score bands (≥70 Low, 40–69 Moderate, <40 High), not tertiles of this project's own sample, since CFPB itself publishes no official categorical cutoff.
- **Feature leakage excluded.** financial_wellbeing_score was deliberately removed from the modeling feature set, since it is a near-deterministic function of the label itself — leaving it in produced a suspicious, meaningless perfect score during early testing.
- **Time-series features not built.** “Spending growth rate” and “savings consistency” from the original proposal were not implemented, since this is cross-sectional (single-snapshot) data rather than multi-month data.
- **Model choice.** Logistic Regression was selected over a marginally higher-scoring SVM, prioritising interpretability and inference speed; the roughly one-point gap is within noise on the test set size used.
- **No persistence layer.** Every prediction is stateless by design — there is no database and nothing entered is stored between requests.
- **3D visual enhancement.** An animated 3D weather-scene enhancement was explored as a visual upgrade but had not yet cleared the project's own visual-quality bar at time of writing. The static SVG icon version shown in this document (Figures 2–3) is the current, shipped version.
- **Not yet done (proposal-level activities).** Usability testing with real users and the full academic dissertation write-up are called for by the original research proposal but sit outside this software artifact's scope (see Section 1).

PART II

Setup Guide

10. Prerequisites
11. Backend & Frontend Setup
12. Running the Application
13. Troubleshooting

10. Prerequisites

Requirement	Notes
Python 3.10 or newer	With venv support. Runs both the ML pipeline and the FastAPI server.
A modern web browser	Chrome, Edge, or Firefox recommended, for the Three.js-enhanced weather visualization (falls back gracefully if unavailable).
No database required	All data is file-based — CSV plus joblib model artifacts.
No Node.js / npm required to run	The frontend is static HTML/CSS/JS; Three.js loads directly from a CDN in the browser. Node is only used by the project's own test scripts, not to run the app.

11. Backend & Frontend Setup

One process serves both the API and the frontend, so there is a single setup path:

1. Open a terminal in the project's root directory.
2. Create and activate a virtual environment:

```
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS/Linux:
source venv/bin/activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Confirm the trained model artifacts exist under models/ (best_model.joblib, scaler.joblib, best_model_metadata.json). If they don't yet, run the training pipeline first (Section 5).
5. Start the server:

```
uvicorn src.app.api:app --reload
```

> **First run**
> Model loading happens once at server startup, so the very first request after starting the server may be marginally slower than every request after it.

12. Running the Application

1. With the server running, open http://localhost:8000 in a browser.
2. Fill in the form and select “Check my financial weather.”
3. Review the result, then select “Check another scenario” to try different inputs.

The interactive API documentation is available at http://localhost:8000/docs, useful for testing the /predict endpoint directly.

13. Troubleshooting

Symptom	Likely Cause	Fix
“Port already in use” starting the server	A previous server instance is still running	Stop the existing process, or run on a different port: uvicorn src.app.api:app --reload --port 8001
Server fails to start, model not found	The training pipeline (Section 5) hasn't been run yet, so models/best_model.joblib doesn't exist	Run the data and model training pipeline first, then restart the server
Form submission returns a validation error	An entered value is outside the accepted range (e.g. age under 18, income ≤ 0)	Check the inline hint under the field — client-side validation mirrors the backend's rules exactly
The weather scene looks flat / static icons only	Expected fallback behaviour — WebGL is unavailable, or the browser/OS has “reduce motion” enabled	Not a bug — the static SVG icons remain fully functional and legible in this mode
Prediction succeeds but factors look generic	Rare edge-case input far outside the training data's typical range	Expected — the model still returns its best estimate, but the explanation is only as informative as how well that input resembles training data

PART III

User Manual

14. Getting Started

15. Filling In Your Information

16. Understanding Your Result

17. What Stood Out & Gentle Suggestions

18. Checking Another Scenario

19. Accessibility & Privacy Notes

14. Getting Started

There's no account, login, or setup needed to use the tool itself. Open the application in a browser and you're ready to begin — the page opens directly to the input form.

15. Filling In Your Information

The form is grouped into five short sections:

- **About you —** age, employment status, and number of dependents.
- **Income & expenditure —** monthly income, total expenditure, essential vs. non-essential spending, and what you planned to spend.
- **Category breakdown —** how spending splits across housing, food, transport, healthcare, education, shopping, entertainment, subscriptions, and other categories.
- **Debt & savings —** monthly debt repayment, number of financial commitments, and current savings.
- **Spending behavior —** impulse and online purchase frequency, how often you check your balance, and how your spending changes during difficult periods.

Each field has an inline hint explaining what it's asking for. Fields are validated as you type, so any out-of-range value is flagged before you submit.

16. Understanding Your Result

After submitting, the form gives way to a result showing one of three states:

- **Clear** — Low financial stress. Your finances currently look steady.
- **Changeable** — Moderate financial stress. Some ups and downs worth keeping an eye on.
- **Stormy** — High financial stress. Worth addressing gently, not alarmingly.

Underneath, a confidence breakdown shows the model's probability for all three states — useful when a result is close, since a 55% Moderate / 40% Stormy split tells a different story than a 96% Stormy result.

17. What Stood Out & Gentle Suggestions

Two panels accompany every result:

- **What stood out —** the three factors that contributed most to your specific result, in plain language (e.g. “High debt-to-income ratio”).
- **A few gentle suggestions —** general, non-professional-advice suggestions tied to those specific factors, not generic financial advice.

A disclaimer is always shown alongside the result: this is an automated estimate based on spending patterns, not a financial or psychological diagnosis.

18. Checking Another Scenario

Select “Check another scenario” at the bottom of the results view to return to a blank form and try different numbers — useful for seeing how a change (like increasing savings or reducing a specific category) might shift the result.

19. Accessibility & Privacy Notes

- **Reduced motion —** if your device has “reduce motion” enabled, the interface automatically uses static icons and skips all animation.
- **Keyboard and screen readers —** every field has a visible focus state and a properly associated label.
- **Mobile —** fully usable down to 375px screen width.
- **Privacy —** nothing entered into the form is stored or shared. There are no accounts and no database — each prediction is calculated and returned without being saved anywhere.

That's it

This covers every part of the application. If anything behaves unexpectedly, the Troubleshooting table in Part II is a good first stop.